

CoderAgent (仮称)

要件定義書

Version : 1.0.0

対象 : Manus AI実装用

形式 : Markdown

Part 1

プロジェクト概要・基本設計

1. プロジェクト概要

1.1 プロジェクト名

正式名称 :

CoderAgent

※名称は仮称。正式公開時に変更可能。

1.2 プロジェクト概要

CoderAgentは、Discord上で利用可能なAIコード開発エージェント Botである。

CLIベースで提供されているClaude Codeのような開発体験を、Discord Bot形式へ移植する。

ユーザーはDiscordのチャット上でAIエージェントと対話することで、

- コード生成
- コード修正
- ファイル作成
- プロジェクト構築
- 開発相談

などを実行できる。

従来CLI操作が必要だったAI開発環境をDiscordへ提供することで、PCを持たないスマートフォンユーザーでも利用可能にする。

2. 開発背景

2.1 現在存在する課題

現在、高性能 AI コードエージェントを利用する場合、以下の問題が存在する。

問題 1

API 料金

高性能モデルを利用する場合、API 利用料金が高額になる。

個人開発者や初心者にとって継続利用の障壁となる。

問題 2

CLI 操作が必要

Claude Code などの開発エージェントは CLI 操作を前提としている。

そのため、

- ターミナル環境
- PC
- コマンド操作知識

が必要になる。

スマートフォンユーザーやプログラミング初心者には利用ハードルが高い。

問題 3

AI 開発環境構築の難しさ

AI を利用した開発では、

- API 設定
- Python 環境構築
- Node 環境構築
- CLI 設定
- ファイル管理

など複数の準備が必要。

CoderAgent では Discord だけで利用できる環境を提供する。

3. 目的

3.1 メイン目的

Discord 上で利用可能な AI コード開発エージェントを提供する。

3.2 達成目標

ユーザーが以下の流れで開発できる状態を作る。

Discord 参加

↓

API キー設定

↓

Coding 開始

↓

AI と会話

↓

コード生成

↓

ファイル取得

↓

開発完了

4. 想定利用者

初期ターゲット

特定 Discord サーバー利用者。

将来的ターゲット

- プログラミング初心者
 - 個人開発者
 - 学生
 - AI 開発ツール利用者
 - Discord コミュニティ
-

5. MVP（初期実装範囲）

MVP 完成条件

以下を満たした時点で初期版完成とする。

必須機能

1. ユーザーが API キーを登録できる
-

2.

AIコーディングセッションを開始できる

利用方法：

Slash Command

`/coding start`

Prefix Command

`!coding start`

3.

専用CodingRoomを自動生成する

条件：

- 使用者本人
- Bot
- 管理者

のみアクセス可能。

4.

AIエージェントとの対話

ユーザーが自然言語で要求を入力。

例：

Discord Botを作って

ログイン機能を追加して

このコードを修正して

AIが処理する。

5.

生成ファイル取得
単一ファイル：
Discord 添付
複数ファイル：
ZIP 化して添付。

6.
セッション終了
/coding end
または
!coding end
終了確認後、
● チャンネル削除
● キャッシュ削除
を実行。

6. 成功指標 (KPI)
初期段階では以下を確認する。

- 技術指標
- Bot稼働率
 - エラー発生率
 - API通信成功率
 - セッション成功率

利用指標

- 登録ユーザー数
 - Coding 開始回数
 - 1ユーザーあたり利用回数
 - 継続利用率
-

7. 技術選定

7.1 開発言語

採用候補

Python 3.12

理由

- Discord Bot 開発実績が豊富
 - AI API との連携が容易
 - 非同期処理に対応
 - 将来的な機械学習・AI 連携との相性が良い
-

7.2 Discord ライブラリ

採用

`discord.py`

理由

- 安定性
- 活発な開発
- Slash Command 対応

- Cog 構成対応
- ドキュメント量

8. 外部 AI API

採用

OpenRouter API

理由

複数 AI モデルを統一 API 形式で利用するため。

将来的に

- Claude
- GPT
- Gemini
- その他 LLM

への切り替えが容易。

9. AI モデル選択方式

ユーザーへ大量のモデル一覧は表示しない。

プリセット方式を採用する。

初期モデル候補

高品質モデル

用途：

- 大規模開発
- 複雑な設計

- 高精度コード生成

バランスモデル (推奨)

用途：

- 通常開発
- コード修正
- 一般利用

低コストモデル

用途：

- 簡単な質問
- 軽量コード生成
- 学習用途

※内部モデルIDは設定ファイルまたはDB管理。
モデル変更時にコード修正不要な設計とする。

10. システム全体構成

概要

Discord User

|



Discord Bot
(discord.py)



Command Layer
(cogs)



Service Layer
(modules)



Database

AI API

PostgreSQL

OpenRouter



File Storage

11. 基本設計方針

最重要事項

本システムは将来的な拡張を前提とする。

設計原則

責務分離

Discord イベント処理と実際の処理ロジックを分離する。

拡張性

将来的に以下を追加可能な構造にする。

- Web 管理画面
 - 有料プラン
 - 使用量管理
 - 複数 AI プロバイダー対応
 - Marketplace
 - API 公開
-

保守性

機能追加時に既存コードへの影響を最小化する。

12. 開発環境

初期実行環境

Manus AI 環境。

将来移行先

VPS。

OS

指定なし。

Linux 環境を想定。

13. 納品物

必須：

GitHub Repository

Manus Project

README.md

.env.example

config.example.json

requirements.txt

14. 機能要件概要

CoderAgent は以下の主要機能を提供する。

MVP 実装対象

機能	優先度
API キー設定	Must

Codingセッション開始	Must
CodingRoom生成	Must
AIエージェント対話	Must
ファイル生成	Must
ファイル提出	Must
セッション終了	Must
サーバー設定	Must
パネル操作	Must
利用制限	Must
ログ管理	Must

15. コマンド仕様

15.1 コマンド方式

CoderAgentでは以下2種類のコマンド方式を採用する。

Slash Command

Discord 標準コマンド。

例：

/coding start

Prefix Command

Slash Commandが利用できない環境向け。

初期Prefix：

！

例：

!coding start

15.2 コマンド共通設計

Slash CommandとPrefix Commandで処理を分けない。

処理構造：

Discord Event

↓

Command Handler

↓

Permission Check

↓

Service Layer

↓

Database/API

同一Serviceを利用する。

16. /setting仕様

目的

ユーザー自身のAI利用設定を管理する。

利用者

一般ユーザー

表示方式

Embed + Select Menu形式。

機能

設定可能項目：

- OpenRouter APIキー登録
 - 利用モデル変更
 - 使用状況確認
 - APIキー削除
-

16.1 APIキー登録

操作：

/setting

↓

Select Menu

↓

API Key 設定

↓

Modal入力

↓

保存

保存処理

入力されたAPIキーはそのまま保存しない。

処理：

User Input



AES 暗号化



Database 保存



保存データ

保存：

- User ID
- 暗号化 API キー
- 登録日時
- 更新日時



注意事項

API キーは以下へ出力禁止。

- Discord メッセージ
- ログ
- エラー通知
- Console



17. /coding start 仕様

目的

AI コーディングセッションを開始する。

実行条件

以下を満たす必要がある。

- API キー登録済み
- 利用制限以内
- Bot 権限あり

処理フロー

ユーザー

↓

/coding start

↓

API キー確認

↓

利用制限確認

↓

CodingRoom 生成

↓

Session 作成

↓

AI Agent 起動

↓

対話開始

18. CodingRoom仕様

目的

ユーザーごとの完全分離された開発環境を提供する。

チャンネル作成

形式：

- coding-0001
- coding-0002
- coding-0003

権限

アクセス可能：

- 実行ユーザー
- Bot
- サーバー管理者

アクセス不可

- 他ユーザー

- 権限を持たないメンバー

18.1 CodingRoom 管理

生成時に以下を保存する。

Guild ID

Channel ID

User ID

Session ID

Conversation ID

Created Time

Last Activity

Model

Status

19. セッション管理

目的

ユーザー間の会話混在を防止する。

セッション識別

以下を組み合わせで管理。

Guild ID

+

Channel ID

+

User ID

+

Session ID

APIキー利用処理

例：

coding-0001

↓

User A

↓

User AのAPIキー取得

↓

OpenRouter 送信

禁止事項：
他ユーザーのAPIキー利用。

20. AIエージェント処理

基本動作

ユーザー入力：
Discord Message
↓
AI Agentへ送信
↓
処理
↓
結果返信

処理例

ユーザー：
Discord Botを作成してください
AI：
設計中...

ファイル作成中...

完成了ました

実行方式

長時間処理を考慮する。

- 必須：
- 非同期処理

- defer 対応
 - timeout 管理
 - exception 処理
-

Discord 応答制御

Discord Interaction の 3 秒制限対策を行う。

禁止：

アプリケーションが応答しませんでした
可能な限り発生させない。

21. AI モデル設定

方針

ユーザーが迷わないよう、モデルを直接大量表示しない。

プリセット方式

例：

高品質

用途：

- 大規模開発
 - 複雑な設計
 - 高精度コード生成
-

バランス（推奨）

用途：

- 一般的な開発
- コード修正
- 日常利用

節約

用途：

- 簡単な生成
- 質問
- 学習用途

内部管理

モデルIDは設定ファイルまたはDB管理。

例：

preset_name

↓

model_id

22. /coding end仕様

目的

現在のCodingRoomを終了する。

処理

実行：

/coding end

または

!coding end

確認画面

Embed + Button

表示：

本当に終了しますか？

終了する

キャンセル

YESの場合

実行：

- チャンネル削除
 - セッション削除
 - キャッシュ削除
 - 一時ファイル削除
-

NOの場合

実行：

- 確認メッセージ削除
- セッション維持

注意

ユーザーAPIキーは削除しない。

23. 自動キャッシュ削除

条件

3日間更新なし。

処理

対象：

- 会話履歴
 - 一時ファイル
 - セッション情報
-

削除対象外：

- ユーザーAPIキー
-

24. /coding panel仕様

目的

Slash Commandが利用できないユーザー向けUI。

表示内容

Embed :

- CoderAgent 説明
- 利用方法
- 注意事項

操作 UI

Select Menu 方式。

選択項目 :

 **Coding Start**
→ コーディング開始

 **Project List**
→ 自分のプロジェクト一覧表示

25. /coding list 仕様

目的

現在または過去のプロジェクト一覧表示。

表示内容

例：
Project

coding-0001

状態：
Active

作成：
2026/07/14

使用モデル：
Claude Sonnet

26. /config 仕様
対象
サーバー管理者。

目的
サーバー単位の設定。

初期設定項目
Coding カテゴリ設定
指定カテゴリ内に CodingRoom を作成。

利用制限

設定例：

- 1ユーザー最大同時チャンネル数
- サーバー全体利用人数

UI

Embed + Select Menu

入力：

Modal

選択：

Button / Select Menu

27. サーバー追加時処理

Botが新規サーバーへ参加した場合。

実行内容

サーバーオーナーへDM送信。

内容：

- 参加への感謝
- CoderAgent 説明
- 初期設定案内
- 設定コマンド案内

28. 利用制限

ユーザー制限

初期設定：

1日50メッセージ

リセット

毎日0:00。

Rate Limit

設定：

8秒に1回

29. 追加予定コマンド

/coding info

現在セッション情報表示。

表示：

- 使用モデル
 - 開始日時
 - ファイル数
 - 使用量
-

/coding export

現在プロジェクトをZIP化。

/coding rename

プロジェクト名変更。

/coding delete

プロジェクト削除。

/health

Bot 状態確認。

表示：

- Bot 稼働状況
- DB 接続
- API 接続

/stats

Bot 統計。

表示：

- サーバー数
 - ユーザー数
 - 利用回数
 - 稼働セッション数
-

/shutdown

Bot Owner 専用。

Bot 安全停止。

30. データ保存方針

30.1 採用データベース

推奨

PostgreSQL

採用理由

- 将来的な Web 管理画面对応が容易
 - ユーザー数増加に対応可能
 - リレーション管理に適している
 - JSON 型データ保存にも対応可能
-

JSON 利用について

一部柔軟なデータは PostgreSQL の JSONB 型を利用する。

例：

- AI 設定
 - メタデータ
 - 拡張設定
-

31. データベース設計

31.1 users テーブル

目的

ユーザー情報管理。

カラム	型	説明
id	BIGSERIAL	内部ID
discord_user_id	VARCHAR	Discord User ID
encrypted_api_key	TEXT	暗号化 API キー
api_provider	VARCHAR	AI Provider
model_preset	VARCHAR	使用モデル設定
created_at	TIMESTAMP	登録日時
updated_at	TIMESTAMP	更新日時

制約

discord_user_id:
UNIQUE

31.2 guilds テーブル

目的

Discord サーバー設定管理。

カラム	型	説明
-----	---	----

id	BIGSERIAL	内部ID
guild_id	VARCHAR	Discord Guild ID
owner_id	VARCHAR	サーバーオーナー
coding_category_id	VARCHAR	作成カテゴリ
user_limit	INTEGER	利用人数制限
channel_limit	INTEGER	同時チャンネル制限
created_at	TIMESTAMP	登録日時

31.3 sessions テーブル

目的

CodingRoom 管理。

カラム	型	説明
id	UUID	Session ID
guild_id	VARCHAR	Guild ID
channel_id	VARCHAR	Discord Channel ID
user_id	VARCHAR	利用者
conversation_id	VARCHAR	AI 会話 ID
model	VARCHAR	使用モデル
status	VARCHAR	状態
created_at	TIMESTAMP	開始日時
last_activity	TIMESTAMP	最終更新

status例

ACTIVE

ENDED

TIMEOUT

ERROR

31.4 projects テーブル

目的

生成プロジェクト管理。

カラム	型	説明
id	UUID	Project ID
session_id	UUID	Session ID
project_name	VARCHAR	名前
file_count	INTEGER	ファイル数
metadata	JSONB	詳細情報
created_at	TIMESTAMP	作成日時
updated_at	TIMESTAMP	更新日時

31.5 messages テーブル

目的

会話履歴管理。

カラム	型	説明
id	BIGSERIAL	ID
session_id	UUID	Session
role	VARCHAR	user/assistant
content	TEXT	内容
token_usage	INTEGER	Token 数
created_at	TIMESTAMP	時間

保存方針

通常利用中のみ保持。

終了時削除。

31.6 usage_logs テーブル

目的

利用統計。

カラム	型	説明
-----	---	----

id	BIGSERIAL	ID
user_id	VARCHAR	User
model	VARCHAR	Model
token_input	INTEGER	入力 Token
token_output	INTEGER	出力 Token
estimated_cost	DECIMAL	推定料金
created_at	TIMESTAMP	日時

31.7 system_logs テーブル

目的

システムログ。

保存内容：

- エラー
- Bot 起動
- Bot 終了
- サーバー参加
- サーバー退出
- API 失敗
- 権限エラー

32. ファイル保存設計

32.1 保存構造

推奨：

storage/

```
└─ users/
    └─ {discord_user_id}/
        └─ projects/
            └─ {project_id}/
                └─ files/
                    └─ main.py
                    └─ config.py
                └─ metadata.json
```

32.2 metadata.json

保存内容：

```
{
  "project_id": "",
  "user_id": "",
  "guild_id": "",
  "created_at": "",
  "updated_at": "",
  "model": "",
  "files": []
}
```

33. API キー暗号化設計

必須要件

API キーを平文保存しない。

保存フロー

User 入力



AES-256 暗号化



Database 保存

利用時

Database 取得



復号



OpenRouter 送信



メモリ破棄

禁止事項

以下への出力禁止。

- Log
- Exception Message
- Discord Message
- Debug Output

34. ディレクトリ構成

採用方式

Cog + Module 方式。

推奨構成

CoderAgent/

└─ main.py

└─ config/

│ └─ config.py

│ └─ constants.py

└─ cogs/

│ └─ setting.py

│ └─ coding.py

│ └─ config.py

| |— admin.py

|— modules/

| |— ai/

| | |— openrouter.py
| | |— agent.py

| |— database/

| | |— models.py
| | |— repository.py

| |— security/

| | |— encryption.py

| |— session/

| | |— manager.py

| |— file/

| | |— manager.py

|— storage/

|— logs/

|— tests/

|— requirements.txt

└─ README.md

└─ .env.example

35. 権限管理

基本方針

全コマンドで権限チェックを必須化。

権限レベル

User

利用者。

可能：

- setting
- coding 開始
- coding 操作

Server Administrator

可能：

- config
- 利用制限設定
- サーバー設定変更

Bot Owner

可能：

- shutdown
- stats
- debug
- health

36. ログ設計

保存対象

ユーザー操作ログ

例：

User A started coding session

システムログ

例：

Database connection failed

APIログ

保存：

- Provider
- Model
- Token 数
- 処理時間

禁止：

API キー

Discord イベント

保存：

- Bot Join
- Bot Leave
- Permission Error

37. エラー処理設計

基本方針

Discord 標準エラー：

アプリケーションが応答しませんでした
を極力発生させない。

37.1 Discord 処理エラー

対応：

`defer()`

↓

処理

↓

followup 送信

37.2 APIエラー

例：

- Timeout
- Rate Limit
- Invalid API Key

処理：

ユーザーへ説明。

例：

OpenRouter API への接続に失敗しました。
API キーまたは通信状況を確認してください。

37.3 内部エラー

処理：

1. Log 保存
 2. User へ簡易通知
 3. Bot Owner へ DM 通知
-

通知内容：

- 発生日時
 - Error 種類
 - Traceback
 - User
 - Guild
 - Session
-

38. 非同期処理要件

必須。

理由：

AI 処理は長時間になるため。

禁止：
メインイベントループを停止する処理。

対象：

- API通信
- ファイル生成
- ZIP 作成
- DB 処理

39. キャッシュ管理

対象

- Conversation
- Session 情報
- 一時ファイル

削除条件

手動終了
即削除。

自動削除

3日間更新なし。

永続保存

対象：

- User API 設定
 - 基本設定
-

40. セキュリティ要件

必須対応

- API キー暗号化
 - 権限チェック
 - SQL Injection 対策
 - Token 秘匿
 - 入力値検証
 - Rate Limit
 - Exception 情報秘匿
-

41. 拡張性設計

将来追加予定：

Web 管理画面

対応可能にする。

必要：

- DB 中心設計
 - Discord 処理分離
 - API Layer 準備
-

有料プラン

対応可能にする。

考慮：

- Usage 管理
- User Plan 管理
- Feature Flag

42. コーディング規約

42.1 基本方針

本プロジェクトは将来的な機能追加を前提とする。

そのため、

- 可読性
- 保守性
- 拡張性
- テスト容易性

を優先した設計を行う。

42.2 言語規約

Python

推奨バージョン：

Python 3.12

互換：

Python 3.11 以上

42.3 コード品質

以下を採用する。

Formatter

Black

Lint

Ruff

Import 整理

isort

型安全

型ヒントを必須とする。

例：

```
async def create_session(
    user_id: int
) -> Session:
```

42.4 命名規則

Class

PascalCase

例：

SessionManager

Function

snake_case

例：

```
create_session()
```

Constant

大文字スネークケース

例：

```
MAX_MESSAGE_LIMIT
```

43. 設計原則

SOLID 原則

採用する。

Single Responsibility Principle

1つのクラス・モジュールは1つの責務のみ持つ。

例：

悪い例：

```
coding.py
```

Discord 処理

AI 処理

DB 処理

暗号化

↓

禁止。

良い例：

`coding.py`

↓

`SessionManager`

↓

`AIService`

↓

`DatabaseRepository`

44. レイヤー設計

Command Layer

場所：

`cogs/`

責務：

- Discord イベント取得
- 入力受付
- 権限確認
- Service 呼び出し

Service Layer

場所：

`modules/`

責務：

- ビジネスロジック
- AI 処理
- セッション管理
- ファイル処理

Repository Layer

責務：

- DB アクセス

禁止事項

Cog 内に以下を書かない。

- SQL
- AI API 通信
- 暗号化処理
- ファイル操作

45. 環境設定管理

秘密情報

以下は禁止。

TOKEN = "xxxxxx"

保存場所

推奨：

.env

または
config.json

対象：

- Discord Token
 - Encryption Key
 - Database URL
 - OpenRouter 設定
-

46. GitHub 運用 Repository

作成する。

推奨構成

main

develop

feature/*

46.1 Commit 規則

例：

feat:

新機能追加

fix:
バグ修正

refactor:
コード整理

docs:
ドキュメント変更

例：
feat: add coding session manager

47. README 要件

README.md を必ず作成する。

内容

プロジェクト概要

CoderAgent 説明。

Features

機能一覧。



Installation

環境構築方法。



Configuration

設定方法。



Commands

コマンド一覧。



Directory Structure

構成説明。



Development

開発方法。



License

ライセンス。

48. ライセンス

正式公開時：
独自ライセンスを作成する。

初期：
内部利用。

ライセンス内容には以下を考慮する。

- 商用利用条件
 - 再配布条件
 - 改変条件
 - API キー管理責任
 - 禁止事項
-

49. テスト方針

初期テスト
開発者による手動テスト。

確認項目

49.1 Bot 起動

確認：

- Discord 接続成功
 - Cog 読み込み成功
 - DB 接続成功
-

49.2 API キー登録

確認：

- 保存成功
 - 暗号化確認
 - 復号確認
-

49.3 Coding 開始

確認：

- チャンネル作成
 - 権限設定
 - Session 作成
-

49.4 AI 通信

確認：

- 正常応答
- Timeout
- API エラー

49.5 ファイル生成

確認：

- 単一ファイル送信
- 複数ファイルZIP化

49.6 終了処理

確認：

- 確認UI表示
- YES処理
- NO処理
- キャッシュ削除

49.7 権限確認

確認：

一般ユーザー：

管理者コマンド不可

管理者：

設定変更可能

Bot Owner：

管理機能可能

50. Manus 向け実装指示

以下を最優先事項として実装する。

50.1 仕様遵守

仕様書に存在しない機能を勝手に追加しない。

改善案がある場合：

実装前に提案事項として提示する。

50.2 実装優先順位

以下順序で開発する。

Phase 1

Bot 基盤

- Discord 接続
- Cog 構成
- Config 管理
- Logging

Phase 2

ユーザー管理

- DB 構築
 - API キー登録
 - 暗号化
-

Phase 3

Coding機能

- Session 管理
 - CodingRoom 生成
 - AI 通信
-

Phase 4

ファイル管理

- 保存
 - ZIP 化
 - 添付
-

Phase 5

管理機能

- Config
 - Stats
 - Health
 - Shutdown
-

Phase 6

品質改善

- Error Handling
 - Refactoring
 - README
-

50.3 エラー処理優先

エラー発生時、
Discord 標準エラーを表示させない。
必ず：

1. Catch
2. Log
3. User通知
4. Owner 通知（必要時）

を行う。

50.4 AI エージェント処理

長時間処理では、
必ず：

`defer()`

↓

Background Task

↓

Progress Update

↓

Complete Response

の流れにする。

50.5 セキュリティ優先

特に以下を厳守。

- API キー漏洩防止
- 権限チェック
- User 分離

- Session 分離
 - Log への秘密情報出力禁止
-

51. 将来ロードマップ

Phase 1

MVP

完成内容：

- Discord Bot
 - AI Coding
 - ファイル生成
 - セッション管理
-

Phase 2

利便性向上

追加予定：

- Web 管理画面
 - 利用統計
 - プロジェクト管理
-

Phase 3

SaaS 化

追加予定：

- 有料プラン
- 利用量管理

- 決済連携
 - ユーザー管理
-

Phase 4

エコシステム化

追加候補：

- Plugin 機能
 - Marketplace
 - API 公開
 - チーム開発機能
-

52. 完成基準

CoderAgent は以下を満たした場合、MVP 完成とする。

機能面

- ☒ Discord から利用可能
 - ☒ AI と自然言語で対話可能
 - ☒ コード生成可能
 - ☒ ファイル取得可能
 - ☒ ユーザーごとに環境分離
 - ☒ API キー安全管理
 - ☒ エラー通知可能
-

品質面

- ☒ 拡張可能な構造

- ☑ README 完備
 - ☑ GitHub 管理
 - ☑ 設定分離
 - ☑ ログ管理
 - ☑ セキュリティ対策済み
-

53. Manus への最終指示

このプロジェクトでは、単なる Discord Bot ではなく、将来的な AI 開発プラットフォームの基盤となることを想定する。

そのため、

- 動作すること
- 拡張できること
- 保守できること
- 安全であること

を優先する。

短期的な実装速度のみを優先せず、将来の Web 管理画面・課金機能・ユーザー増加を考慮した設計を行う。仕様書に記載されている内容を基準とし、不明点は推測で実装せず確認または TODO として管理する。
